

一、实验目的

- 1、深度领会知识蒸馏技术的原理与运行机制，明晰硬标签、软标签在知识迁移进程中的具体作用。
- 2、熟练掌握运用大模型生成信息抽取标签的方法，学会对大模型产出的标注数据进行妥善处理与分析。
- 3、借助知识蒸馏技术将大模型知识迁移至小模型，训练出能够高效执行信息抽取任务的轻量化模型。
- 4、在特定数据集上对训练完成的模型展开全面评估，深入剖析知识蒸馏技术对信息抽取模型性能所产生的影响。

二、实验环境

- 1、硬件环境：微机
- 2、软件环境：Windows 操作系统；python 集成开发环境。

三、实验原理

1、知识蒸馏技术

在自然语言处理范畴内，信息抽取任务旨在从各类文本数据中精准提取关键信息，例如实体、关系以及事件等。此任务在智能问答系统、舆情分析、信息检索等诸多领域均有广泛应用，具有极为重要的价值。大模型在信息抽取任务中展现出了卓越的性能，但因其规模巨大，在实际部署与应用时面临着计算资源紧张以及推理速度迟缓等难题。

知识蒸馏技术作为一种高效的模型压缩与优化手段，能够将大模型（即教师模型）所蕴含的知识迁移至小模型（即学生模型），使小模型在维持较高性能的同时，有效降低计算成本，为解决上述问题提供了可行路径。本实验专注于基于大模型知识蒸馏的信息抽取，深入探究该技术在这一领域的应用价值与优化方向。

2、命名实体识别

命名实体识别（Named Entity Recognition, NER）是自然语言处理（NLP）中的一项核心技术，旨在从非结构化文本中识别并分类预定义的实体类别（如人名、地点、组织机构、时间、日期、货币等）。

四、实验重点

1、知识蒸馏技术应用

熟练掌握知识蒸馏的核心原理和实现方式，能够根据信息抽取任务的特点，合理设计并调整蒸馏策略，实现大模型知识向小模型的有效迁移。

2、模型训练与调优

掌握大模型标注数据的处理方法，以及小模型的训练和调优技巧。通过调整超参数、优化损失函数等手段，提升小模型在信息抽取任务中的性能。

3、实验结果分析

学会运用恰当的评估指标对模型性能进行全面、准确的评估。能够深入剖析实验结果，总结知识蒸馏技术在信息抽取中的应用规律与效果。

五、实验难点

1、蒸馏策略优化

确定最优的知识蒸馏策略颇具挑战，需要深入理解软标签和硬标签的作用机理，以及温度参数等对蒸馏效果的影响。在众多参数组合中探寻最优解，需要开展大量实验并进行细致分析。

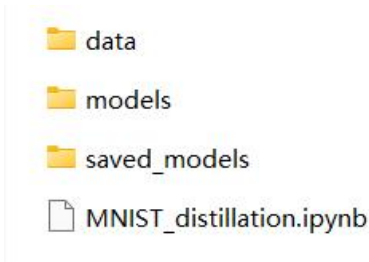
2、模型性能平衡

在知识蒸馏过程中，需要在小模型的计算资源消耗和性能之间寻求平衡。既要确保小模型能够充分汲取大模型的知识，又要避免过度拟合，保证模型在不同数据集上具备良好的泛化能力。

六、参考代码说明

1. MNIST

MNIST 文件夹为手写数字识别示例，data 文件夹为原始数据（ubyte 格式），MNIST_distillation.ipynb 为示例代码，models 和 saved_models 为代码运行时训练并保存的学生模型和教师模型。



2. CONLL-NER

CONLL-NER 文件夹为命名实体识别示例，包含：

（1）数据集

其中，conll2003-txt 文件夹中为 txt 文件类型的 conll2003 数据集，包含 train.txt、test.txt、valid.txt，数据形式为分词后的 BIO 格式标签：

Data_for_training_and_evaluating.xlsx 为 excel 文件类型，包含 4 张表，分别是：

Sheet 名称	说明
pure_conll_distilled	大模型标注的 conll2003 数据标签
conll+bbc_distilled	大模型标注的 conll2003+bbc 新闻数据集数据标签
pure_original	原始带标签的数据集
test_set	带标签的测试数据集

（2）代码文件

Sheet 名称	说明
deepseek_process.ipynb	利用 deepseek 进行句子的实体识别，批量生成 BIO 标签
bert_original_data_training.ipynb	利用真实标签训练和测试 BERT 模型
bert_distillation.ipynb	大模型标注的 conll 数据+真实标签混合训练的 bert 模型

（3）bert_distillation.ipynb 代码运行结束后，保存的训练好的模型（参数）。

conll2003-txt	2025/3/31 17:06	文件夹	
bert_distillation.ipynb	2025/4/4 21:22	IPYNB 文件	40 KB
bert_original_data_training.ipynb	2025/3/31 17:07	IPYNB 文件	34 KB
Data_for_training_and_evaluating.xlsx	2025/3/28 12:02	XLSX 工作表	499 KB
deepseek_process.ipynb	2025/3/28 15:34	IPYNB 文件	29 KB
ner_model.pth	2025/4/4 13:17	PTH 文件	425,462 KB

七、实验内容（展示数据集、实验代码、参数设置、权重分配、模型效果、结果分析等内容）

1、数据集选择与预处理：自主挑选适用于信息抽取的数据集，数据集来源既可以是公开数据集，也可以是自行采集的多样化文本数据。对数据实施清洗、标注转换等预处理操作，为后续实验奠定基础。

- 2、大模型与小模型选择：择取合适的大模型和小模型作为教师和学生模型。
- 3、蒸馏策略设计：确定知识蒸馏的具体策略，涵盖软标签和硬标签的权重分配、温度参数的设定等。思考如何依据实验需求灵活调整这些参数，以达成最佳的知识迁移效果。
- 4、实验对比设计：设计对比实验，对比仅采用硬标签训练小模型、运用知识蒸馏训练小模型以及直接运用大模型进行信息抽取的性能差异。
- 5、结果分析与优化：深入分析实验结果，探讨知识蒸馏技术在信息抽取中的优势与不足；研究不同蒸馏策略和参数设置对模型性能的影响，找出影响知识迁移效果的关键因素。依据实验结果和分析，提出对知识蒸馏过程的优化方案。例如，调整软标签和硬标签的生成方式、改进损失函数的设计、尝试不同的小模型结构等。

整体思路梳理

【方案一】首先需要使用 Ollama 服务调用 deepseek-r1: 1.5b 模型，构建提示词，进行对话。在实例代码中是要求输出文本标注的 BIO 标签，但是考虑到蒸馏需要使用软标签，也就是教师模型经过 softmax 后的概率分布，所以我在这里选用了让模型输出每个标签的置信度作为概率分布情况，然后进行归一化，再存储为 csv 文件，方便后续学生模型训练时计算软标签损失。

然后再训练学生模型，使用多个软标签和硬标签同时训练。此外再使用硬标签单独训练一个学生模型，对比两者的性能差异。

但是后续在使用 Ollama 服务和 deepseek-r1: 1.5b 模型进行对话时，由于电脑的硬件条件有限，所以无法对所有数据样本进行处理。并且 1.5b 模型的分类效果也并不是很好，所以最后使用了方案二——在线的蒸馏训练实验。

【方案二】由于电脑的硬件条件有限，考虑到方案一需要较大的计算资源，因此准备了方案二：使用 CIFAR-10 数据集，教师模型使用 Resnet18 神经网络模型用于处理图像数据，学生模型使用自主架构的卷积层、池化层、全连接层神经网络进行训练。并设计对照试验，将使用知识蒸馏法训练的学生模型与单独训练的学生模型进行对比，比较差异。分析关键超参数——蒸馏温度、alpha 对模型训练效果的影响。

（一）数据集选择

1.文本分类数据集【方案一】

后续考虑到 CIFAR-10 数据集是图像数据，如果用于训练模型，参数量和计算量会非常大，所

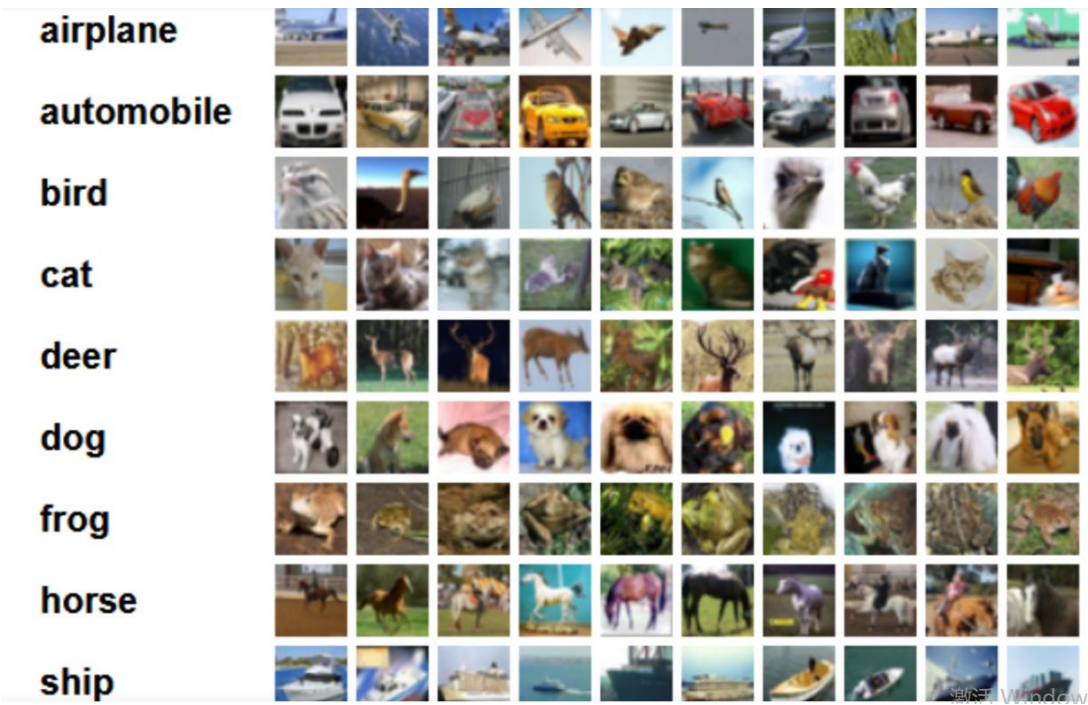
以更换成了文本数据集。选用了 AG_NEWS 文本分类数据集，文本内容是新闻，一共包括四个标签： world、business、science、sports 四个标签。

实验中使用 dataset 包中的 load_dataset 从 HuggingFace 上加载数据集进行处理。

2.图像数据集【方案二】

CIFAR10 数据集共有 60000 个样本，每个样本都是一张 32*32 像素的 RGB 图像（彩色图像），每个 RGB 图像又必定分为 3 个通道（R 通道、G 通道、B 通道）。这 60000 个样本被分成了 50000 个训练样本和 10000 个测试样本。

CIFAR10 数据集有 10 类物体，标签值分别按照 0~9 来区分,他们分别是飞机（ airplane ）、汽车（ automobile ）、鸟（ bird ）、猫（ cat ）、鹿（ deer ）、狗（ dog ）、青蛙（ frog ）、马（ horse ）、船（ ship ）和卡车（ truck ）。



为了提高模型的泛化能力，所以对模型进行翻转、平移的操作，在保证数据基本模式不变的前提下，对训练数据进行增强。

```
def prepare_data():
    transform_train = transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(), # 数据增强策略
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010))
    ])

    transform_test = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010))
    ])
```

加载数据集，如果需要下载到本地就将 download 函数设为 true，否则设置为 false，通过进度条显示下载成功；

接下来，使用如下代码，可以查看数据集总共的类别标签以及对应的样本数量，可以看到总共有 10 类，且每一类都有 5000 个样本：

```
# 分析数据集类别分布 - 适配三个返回值的情况
def analyze_dataset(trainset, testset):
    # 获取类别名称
    classes = trainset.classes
    print(f"数据集中共有 {len(classes)} 个类别: {classes}")

    # 统计训练集各类别样本数
    train_class_counts = {cls: 0 for cls in classes}
    for data in tqdm(trainset, desc="分析训练集"):
        _, label, _ = data # 解包三个返回值
        train_class_counts[classes[label]] += 1

    # 统计测试集各类别样本数
    test_class_counts = {cls: 0 for cls in classes}
    for data in tqdm(testset, desc="分析测试集"):
        _, label, _ = data # 解包三个返回值
        test_class_counts[classes[label]] += 1

    # 打印统计结果
    print("\n训练集样本分布:")
    for cls, count in train_class_counts.items():
        print(f"{cls}: {count} 样本")

    print("\n测试集样本分布:")
    for cls, count in test_class_counts.items():
        print(f"{cls}: {count} 样本")
```



```
数据集中共有 10 个类别: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
分析训练集: 100%|██████████| 50000/50000 [00:51<00:00, 973.56it/s]
分析测试集: 100%|██████████| 50000/50000 [00:49<00:00, 1001.73it/s]
C:\Users\Administrator\AppData\Local\Temp\ipykernel_160832\3316320582.py:45: UserWarning: Glyph 31867 (\N{CJK UNIFIED IDEOGR
plt.tight_layout()
C:\Users\Administrator\AppData\Local\Temp\ipykernel_160832\3316320582.py:45: UserWarning: Glyph 21035 (\N{CJK UNIFIED IDEOGR
plt.tight_layout()

训练集样本分布:
airplane: 5000 样本
automobile: 5000 样本
bird: 5000 样本
cat: 5000 样本
deer: 5000 样本
dog: 5000 样本
frog: 5000 样本
horse: 5000 样本
ship: 5000 样本
truck: 5000 样本

测试集样本分布:
airplane: 5000 样本
automobile: 5000 样本
bird: 5000 样本
cat: 5000 样本
deer: 5000 样本
dog: 5000 样本
frog: 5000 样本
horse: 5000 样本
ship: 5000 样本
truck: 5000 样本
```

激
转5

(二) 方案一

1. Ollama 及模型下载（教师模型）

安装 ollama 服务助手以及相应的 deepseek 大模型后，如果想要在 VScode 中与其对话，则需要
在 python 的虚拟环境中安装 ollama 包，然后导入 chat 模块，尝试调用大模型并进行对话测试，结
果也可以正常输出：

```
from ollama import chat

def annotate(prompt):
    response = chat(
        model='deepseek-r1:1.5b',
        messages=[{'role': 'user', 'content': prompt}])
    return response.message.content.split("</think>\n\n")[-1]

annotate("hello")

[2] ✓ 0.7s

... 'Hello! How can I assist you today? 😊'
```

2. 教师模型信息存取

(1) 编写提示词

由于 AG_NEWS 是英文新闻数据集，所以我在这里使用的是大模型经过翻译后的英文提示词。最终要求模型输入四个标签分别对应的概率值，并且强制要求四个标签对应的概率值和为 1。使用置信度/可信度当做软标签。

```
classification_prompt = """As an expert news analyst, provide probability distribution for this news text:

**Text**: {text}

**Categories**:
1. World - International affairs, politics, disasters
2. Sports - Athletic competitions, players, match results
3. Business - Financial markets, corporate activities
4. Science/Tech - Technological innovations, scientific discoveries

**Requirements**:
1. Analyze text content
2. List key entities
3. Output FOUR probabilities (space-separated) summing to 1.0

**Output Format**:
prob_world prob_sports prob_business prob_sci_tech

Example 1:
Text: "Apple launches new AI-powered chip"
Output: 0.02 0.0 0.28 0.70

Example 2:
Text: "Eurozone leaders reach debt agreement"
Output: 0.85 0.0 0.15 0.0

Now provide probabilities for this text:"""
```

1 ✓ 0.0s

(2) 使用 ollama 进行大模型对话

首先设置在与模型对话获取新闻文本的软标签过程中，需要使用到的超参数，如下：

```
class Config:
    # 数据集参数
    dataset_name = "ag_news"
    text_field = "text" # 数据集新闻文本字段
    label_field = "label" # 数据集新闻标签字段
    label_names = ["World", "Sports", "Business", "Science/Tech"]

    # 模型参数
    ollama_model = "deepseek-r1:1.5b"
    max_retries = 3 # API调用重试次数
    retry_delay = 2 # 重试等待时间（秒）

    # 输出路径
    output_csv = "data/ag_news_labels.csv"

cfg = Config()
```

0.0s

在 anaconda 虚拟环境中安装 ollama，使用 ollama.chat()与模型对话，输入构建好的提示词。用 response 获取大模型输出结果。

```
full_prompt = classification_prompt.format(text="Today is a terrible day for bosses.Trump added 60% tax for China.")
response = ollama.chat(model=cfg.ollama_model,
                        messages=[{'role': 'user', 'content': full_prompt}],
                        options={'temperature': 0.0})
raw_output = response['message']['content']
probs = parse_probabilities(raw_output)
print(raw_output)
print(probs)
```

由于模型先进行思考推理，真正需要的软标签会在最后输出，所以需要构建 parse_probabilities 函数从大模型输出的结尾开始，匹配四个数字（对应四个标签）然后输出。尽管在提示词中要求概率分布和为 1，但是大模型可能会不遵守这一条限制，因此需要进行强制的归一化处理，让概率分布和为“1”。

```
# 增强的解析函数
import re

def parse_probabilities(raw_output):
    try:
        # 去除非数字字符
        cleaned = re.sub(r'^\d\s\.', '', raw_output)

        # 逆序查找最后出现的4个数值
        lines = cleaned.strip().split('\n')
        for line in reversed(lines): #倒叙寻找
            parts = line.strip().split()
            if len(parts) == 4:
                # 验证是否为合法概率值
                probs = list(map(float, parts))
                if all(0 <= p <= 1 for p in probs):
                    # 强制归一化
                    total = sum(probs)
                    if abs(total - 1.0) > 0.001:
                        probs = [p/total for p in probs]
                    return [round(p, 4) for p in probs]

        # 未找到有效数值时返回均匀分布
        return [0.25]*4
    except:
        return [0.25]*4
```

大模型直接的输出结果和剔除了思考推理过程并进行强制归一化的预处理操作后的软标签如下：

```

.. <think>
Alright, let's tackle this probability distribution problem step by step. The task is to analyze the given news text and assign p

First, I need to understand what the text says. It mentions that "Today is a terrible day for bosses. Trump added 60% tax for Chin

Now, looking at the categories:

1. **World - International affairs, politics, disasters**: This category includes events related to international politics or dis

2. **Sports - Athletic competitions, players, match results**: There's no mention of sports here. It's about business-related top

3. **Business - Financial markets, corporate activities**: The text discusses tax additions, which are part of corporate financia

4. **Science/Tech - Technological innovations, scientific discoveries**: There's no mention of technological or scientific elemen

Putting it all together, the probabilities should sum to 1.0. Based on the analysis, World has a high probability (likely around 0.75)

</think>

prob_world prob_sports prob_business prob_science_tech
0.85 0.0 0.28 0.0
[0.7522, 0.0, 0.2478, 0.0]

```

此外，由于原始的 `ag_news` 数据集数据量过大，本机无法支持运行，所以只选取了原数据集中的测试集作为本实验的数据集，保存为 `csv` 格式文件。

接下来需要对数据集中的每一条新闻文本使用 `ollama` 服务和大模型对话，并将处理后的概率分布存入新的 `csv` 文件中，代码如下（这里是试运行，所以只选了 100 条数据）：

```

# 数据处理流程
def generate_labels(dataset):
    # 创建CSV文件
    with open(cfg.output_csv, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        header = ["text", "hard_label"] + cfg.label_names
        writer.writerow(header)

    # 分批处理
    for i in tqdm(range(0, 100, 5), desc="处理中====="):
        batch = dataset[i:i+5]
        batch_rows = []

        for text, hard_label in zip(batch["text"], batch["label"]):
            # 生成提示词
            full_prompt = classification_prompt.format(text=text)
            # print("已生成提示词! =====")

            # 调用模型
            for _ in range(cfg.max_retries):
                try:
                    response = ollama.chat(
                        model=cfg.ollama_model,
                        messages=[{'role': 'user', 'content': full_prompt}],
                        options={'temperature': 0.0}
                    )
                    raw_output = response['message']['content']
                    probs = parse_probabilities(raw_output)
                    break
                except:
                    probs = [0.25]*4
                    time.sleep(cfg.retry_delay)

            # 构建数据行
            batch_rows.append([text, hard_label] + probs)

```

然后批量保存数据到 csv 文件中，文件内容如下（包括原始文本、硬标签以及每个类别标签对应的概率分布）：

```
if __name__ == "__main__":
    generate_labels(dataset)
    print(f"标注数据已保存至: {cfg.output_csv}")

[7]
... 处理中=====: 0%|          | 0/5 [00:00<?, ?it/s]
    已生成提示词! =====
    已完成调用! =====
    已生成提示词! =====
    处理中=====: 20%|██      | 1/5 [00:41<02:44, 41.00s/it]
    已生成提示词! =====
    已完成调用! =====
    已生成提示词! =====
    处理中=====: 40%|████    | 2/5 [01:48<02:49, 56.60s/it]
    已生成提示词! =====
    已完成调用! =====
    已生成提示词! =====
```

	A	B	C	D	E	F
1	text	hard_label	World	Sports	Business	Science/Tec
2	Unions reg	3	0.7143	0.0952	0.1905	0
3	SPACE.com	4	0.5	0	0.3	0.2
4	AP - A com	4	0.2083	0.2083	0.25	0.3333
5	AP - It's	4	1	0	0	0
6	AP - South	4	0.3	0.2	0.5	0
7	The Britis	4	0	0	0.8182	0.1818
8	\\Sven Jæ	4	0.0306	0	0.5612	0.4082
9	\\FOAF/LO	4	0.45	0.25	0.15	0.15
10	Wiltshire	4	0.25	0.25	0.25	0.25
11	In its fir	4	0.7576	0.0202	0.1818	0.0404
12	LOS ANGE	4	0.2105	0.2105	0.3684	0.2105
13	LOS ANGE	4	0.25	0.25	0.25	0.25
14	AMSTERDA	4	0.45	0	0.38	0.17
15	A giant 10	4	0.7	0.15	0	0.15
16	Dolphin gr	4	0.25	0.25	0.25	0.25
17	Tyrannosau	4	0.85	0	0	0.15
18	Jet Propul	4	0.5	0	0	0.5
19	European S	4	0.8947	0	0	0.1053
20	When did l	4	0.5	0	0	0.5
21	update Ear	4	0.974	0.026	0	0
22	By the end	4	0.2478	0	0.6195	0.1327
23	Developers	4	0.25	0.25	0.25	0.25

但是由于数据量过大，即使减少批量处理数据量至 5，仍然运行速度非常慢，当选择了 100 条数据，批量大小为 5 时，任务进度就会卡在 25%。因此该方案在我经历了多次尝试后，最终选择了放弃，使用了方案二。

```
if __name__ == "__main__":
    generate_labels(dataset)
    print(f"标注数据已保存至: {cfg.output_csv}")

[8]
... 处理中=====: 25%|██      | 5/20 [08:11<22:59, 91.95s/it]
```

（三）方案二

使用 CIFAR-10 作为训练的数据集。

然后选用 restnet18 作为大模型（ResNet18 是一个经典的深度卷积神经网络模型，名称来源于网络中包含的 18 个卷积层，可以被用于图像分类）。小模型使用含有 2 个卷积层，1 个平均池化层，1 个全连接层，使用“relu”激活的神经网络。

设计对比实验（单独训练的学生模型 vs.知识蒸馏训练的学生模型），教师模型和单独训练的学生模型都使用反向传播算法更新参数；蒸馏训练的学生模型使用教师模型的软标签和原数据的分类标签进行训练。

最后输出分类报告，评估性能。

1. 定义模型

(1) 教师模型

由于 resnet18 的输入是 224×224 的图像，CIFAR-10 的图像像素比较小，所以需要进行微调，定义的教师模型如下：

```
# 模型定义
# =====
class Teacher(nn.Module):
    def __init__(self):
        super(Teacher, self).__init__()
        self.net = torchvision.models.resnet18(pretrained=False)
        # 调整输入通道数
        self.net.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
        self.net.fc = nn.Linear(512, 10) # CIFAR-10有10个类别

    def forward(self, x):
        return self.net(x)
```

[67] ✓ 0.0s

(2) 学生模型

构建的学生模型类如下，包含两层卷积层，一层平均池化层，一层全连接层：

```
class Student(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Conv2d(3, 16, 3, stride=2, padding=1),
            nn.GELU(),
            nn.BatchNorm2d(16),
            nn.Conv2d(16, 32, 3, stride=2, padding=1),
            nn.GELU(),
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Linear(32, NUM_CLASSES)
        )

    def forward(self, x):
        return self.model(x)
```

✓ 0.0s

2. 训练函数&蒸馏训练函数

(1) 训练函数（教师模型+单独训练的学生模型）

因为该任务属于分类问题，所以使用交叉熵损失函数，学习率设为 1×10^{-3} ，使用 adamw 优化器。

```
# 训练流程
# =====
def train_teacher(model, train_loader, test_loader):
    model.train()
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
    scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=200)
```

使用 tqdm 打印训练过程的进度条，显示训练进度，并使用 val_acc 存储模型在测试集上的准确率。

```
best_acc = 0.0
history = {'train_loss': [], 'val_acc': []}

for epoch in range(EPOCHS):
    model.train()
    total_loss = 0.0

    with tqdm(train_loader, unit="batch", desc=f"Teacher Epoch-{epoch+1}/{EPOCHS}") as pbar:
        for images, labels in pbar:
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            pbar.set_postfix({'loss': f"{loss.item():.4f}"})

    # 验证
    val_acc = evaluate(model, test_loader)
    scheduler.step()

    history['train_loss'].append(total_loss/len(train_loader))
    history['val_acc'].append(val_acc)

    # 保存最佳模型
    if val_acc > best_acc:
        best_acc = val_acc
        torch.save(model.state_dict(), "models/single_student.pth")

    print(f"Epoch-{epoch+1}: Val Acc-{val_acc:.2f}%")

return history
```

最后，为了方便学生模型根据教师模型的输出进行训练，使用如下代码对教师模型进行保存：

```

# 保存最佳模型
if val_acc > best_acc:
    best_acc = val_acc
    torch.save(model.state_dict(), "models/best_teacher.pth")

print(f"Epoch {epoch+1}: Val Acc {val_acc:.2f}%")

return history

```

✓ 0.0s

(2) 知识蒸馏训练函数

向 `distill()` 函数传入训练的学生模型和教师模型，使用 KL 散度和交叉熵损失函数结合作为总损失值。

```

def distill(student, teacher, train_loader, test_loader):
    student.train()
    teacher.eval()

    criterion = nn.KLDivLoss(reduction='batchmean')
    optimizer = optim.AdamW(student.parameters(), lr=1e-3)
    best_acc = 0.0
    history = {'loss': [], 'acc': []}

```

```

for epoch in range(EPOCHS):
    total_loss = 0.0
    correct = 0
    total = 0
    with tqdm(train_loader, unit="batch", desc=f"Distill Epoch {epoch+1}/{EPOCHS}") as pbar:
        for images, labels in pbar:
            optimizer.zero_grad()

            with torch.no_grad():
                teacher_logits = teacher(images)

            student_logits = student(images)

            # 蒸馏损失
            loss_distill = criterion(
                torch.log_softmax(student_logits/TEMPERATURE, dim=1),
                torch.softmax(teacher_logits/TEMPERATURE, dim=1)
            ) * (TEMPERATURE**2)

            # 交叉熵损失
            loss_ce = nn.CrossEntropyLoss()(student_logits, labels)
            loss = ALPHA*loss_distill + (1-ALPHA)*loss_ce #总损失

            loss.backward()
            optimizer.step()

```


3. 模型训练效果

(1) 教师模型

首先加载数据集，然后，实例化一个教师模型对象。传入实例化的教师模型以及训练集和测试集：

```
train_set, test_set = prepare_data()
train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True, num_workers=4)
test_loader = DataLoader(test_set, batch_size=BATCH_SIZE, shuffle=False, num_workers=4)

# 实验1: 单独训练教师模型
print("\n=== Training Teacher ===")
teacher = Teacher()
teacher_hist = train_teacher(teacher, train_loader, test_loader)
```

如下是整体的训练效果，准确率超过 84%。首先因为本机的计算资源有限，因此模型的准确率受限；其次，使用的数据集数据量大，且模式较为复杂，后续可以在计算机硬件条件允许的情况下，选用更为复杂的模型：

```
teacher_hist = train_teacher(teacher, train_loader, test_loader)
```

[27] ✓ 92m 58.4s

...

```
=== Training Teacher ===
Teacher Epoch 1/10: 100%|██████████| 391/391 [07:31<00:00, 1.15s/batch, loss=0.9791]
Epoch 1: Val Acc 57.90%
Teacher Epoch 2/10: 100%|██████████| 391/391 [07:33<00:00, 1.16s/batch, loss=0.7404]
Epoch 2: Val Acc 68.20%
Teacher Epoch 3/10: 100%|██████████| 391/391 [07:37<00:00, 1.17s/batch, loss=0.7877]
Epoch 3: Val Acc 75.89%
Teacher Epoch 4/10: 100%|██████████| 391/391 [07:27<00:00, 1.14s/batch, loss=0.5613]
Epoch 4: Val Acc 73.96%
Teacher Epoch 5/10: 100%|██████████| 391/391 [07:27<00:00, 1.14s/batch, loss=0.5591]
Epoch 5: Val Acc 79.84%
Teacher Epoch 6/10: 100%|██████████| 391/391 [07:27<00:00, 1.14s/batch, loss=0.3709]
Epoch 6: Val Acc 80.90%
Teacher Epoch 7/10: 100%|██████████| 391/391 [08:41<00:00, 1.33s/batch, loss=0.4192]
Epoch 7: Val Acc 81.91%
Teacher Epoch 8/10: 100%|██████████| 391/391 [10:59<00:00, 1.69s/batch, loss=0.3579]
Epoch 8: Val Acc 83.55%
Teacher Epoch 9/10: 100%|██████████| 391/391 [11:02<00:00, 1.69s/batch, loss=0.4413]
Epoch 9: Val Acc 82.51%
Teacher Epoch 10/10: 100%|██████████| 391/391 [11:32<00:00, 1.77s/batch, loss=0.4613]
Epoch 10: Val Acc 84.30%
```

进行单张图片类别预测，对该图片进行预处理，和定义模型时对测试集的处理保持一致（由于为了增强模型的泛化能力，对训练集进行了数据增强，这里的预测不需要）。

然后调用模型，输出每个类别的概率分布以及最终的分类结果，如下：

```

from PIL import Image
def predict_single_image(model, img_path):

    class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
                    'dog', 'frog', 'horse', 'ship', 'truck']

    # 预处理图像
    transform = get_test_transform()
    img = Image.open(img_path).convert('RGB')
    input_tensor = transform(img).unsqueeze(0) # 添加batch维度

    # 执行预测
    with torch.no_grad():
        output = model(input_tensor)
        probabilities = torch.nn.functional.softmax(output[0], dim=0)
        pred_class = torch.argmax(probabilities).item()

```

```

# 可视化结果
plt.figure(figsize=(10, 5))

# 显示原始图像
plt.subplot(1, 2, 1)
plt.imshow(img)
plt.title("Input Image")
plt.axis('off')

# 显示预测概率
plt.subplot(1, 2, 2)
plt.barh(class_names, probabilities.numpy())
plt.title(f"Prediction: {class_names[pred_class]}")
plt.xlabel("Probability")
plt.tight_layout()
plt.show()

```

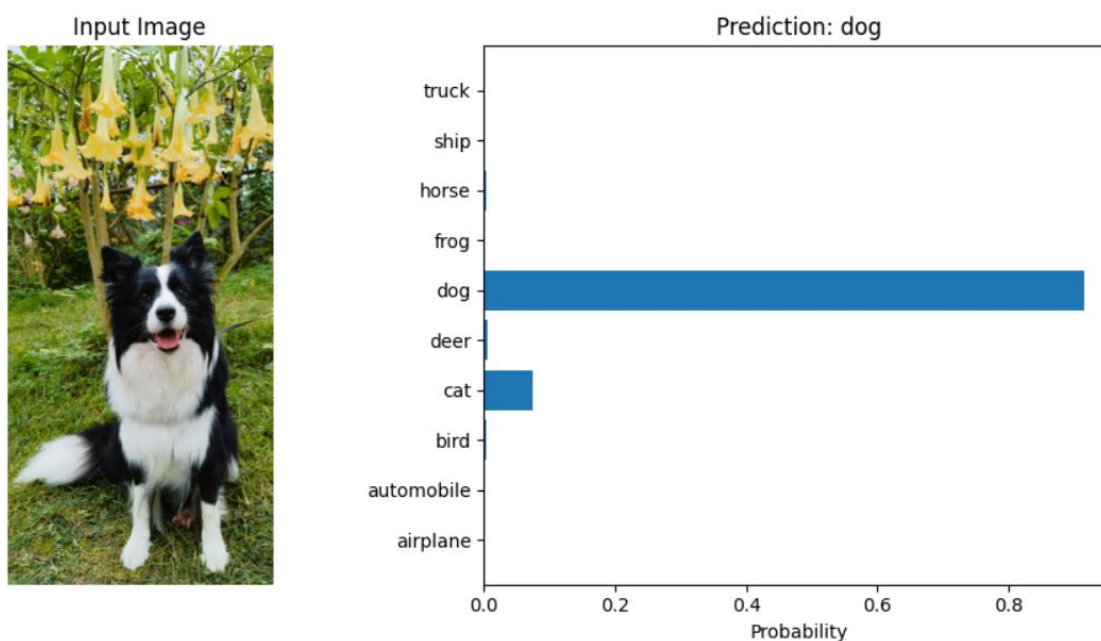
分类预测结果如下，可以看到模型的性能还是比较好的，dog 的概率分布超过了 80%。

```

img_path = "data\dog.jpg" # 图片相对路径
prediction = predict_single_image(teacher, img_path)
print(f"最终预测结果: {prediction}")

```

[129] ✓ 0.3s



(2) 知识蒸馏训练的学生模型

以下是通过知识蒸馏训练的学生模型效果实例（给出了部分的 epochs 训练过程结果），但是可以看到模型的效果并没有像教师模型一样较好，只达到了 52%。推测可能是因为学生模型过于简单，无法抽取出数据中的复杂特征（超参数→temperature: 3, alpha: 0.5）：

```
# 实验2: 知识蒸馏训练学生
print("\n=== Distilling Student ===")
teacher.load_state_dict(torch.load("models/best_teacher.pth"))
student = Student()
student_hist = distill(student, teacher, train_loader, test_loader)
```

[140] ✓ 52m 8.4s

```
Epoch 12: Val Acc 48.86%
Distill Epoch 13/20: 100%|██████████| 391/391 [01:39<00:00, 3.92batch/s, loss=1.7906, acc=49.07%]
Epoch 13: Val Acc 48.36%
Distill Epoch 14/20: 100%|██████████| 391/391 [01:40<00:00, 3.91batch/s, loss=1.9779, acc=49.32%]
Epoch 14: Val Acc 48.39%
Distill Epoch 15/20: 100%|██████████| 391/391 [02:30<00:00, 2.59batch/s, loss=2.0496, acc=50.00%]
Epoch 15: Val Acc 50.27%
Distill Epoch 16/20: 100%|██████████| 391/391 [01:56<00:00, 3.35batch/s, loss=1.7631, acc=50.27%]
Epoch 16: Val Acc 51.05%
Distill Epoch 17/20: 100%|██████████| 391/391 [15:42<00:00, 2.41s/batch, loss=1.7998, acc=50.61%]
Epoch 17: Val Acc 50.40%
Distill Epoch 18/20: 100%|██████████| 391/391 [02:11<00:00, 2.97batch/s, loss=1.7527, acc=50.94%]
Epoch 18: Val Acc 50.71%
Distill Epoch 19/20: 100%|██████████| 391/391 [02:10<00:00, 2.99batch/s, loss=1.9347, acc=51.32%]
Epoch 19: Val Acc 51.46%
Distill Epoch 20/20: 100%|██████████| 391/391 [02:01<00:00, 3.22batch/s, loss=1.7083, acc=51.65%]
Epoch 20: Val Acc 52.03%
```

(3) 单独训练的学生模型

为了对比知识蒸馏训练的学生模型与单独训练的学生模型的区别，同时设计了学生模型单独训练的实验（使用教师模型训练函数）。

以下是单独训练的学生模型效果：

```
# 实验3: 直接训练学生模型（对比基准）
print("\n=== Training Student Directly ===")
baseline_student = Student()
baseline_hist = train_teacher(baseline_student, train_loader, test_loader)
```

✓ 16m 13.4s

可以看到与知识蒸馏训练的模型相比，该模型训练时间更短（16min<52min），但是效果不如知识蒸馏训练的学生模型（只有 51%）。

```

Epoch 12: Val Acc 47.20%
Teacher Epoch 13/20: 100% ██████████ 391/391 [00:51<00:00, 7.62batch/s, loss=1.3821]
Epoch 13: Val Acc 48.40%
Teacher Epoch 14/20: 100% ██████████ 391/391 [00:50<00:00, 7.73batch/s, loss=1.2064]
Epoch 14: Val Acc 50.04%
Teacher Epoch 15/20: 100% ██████████ 391/391 [00:50<00:00, 7.72batch/s, loss=1.1139]
Epoch 15: Val Acc 49.49%
Teacher Epoch 16/20: 100% ██████████ 391/391 [00:51<00:00, 7.65batch/s, loss=1.4721]
Epoch 16: Val Acc 49.22%
Teacher Epoch 17/20: 100% ██████████ 391/391 [00:51<00:00, 7.60batch/s, loss=1.3781]
Epoch 17: Val Acc 50.21%
Teacher Epoch 18/20: 100% ██████████ 391/391 [00:51<00:00, 7.59batch/s, loss=1.4438]
Epoch 18: Val Acc 50.65%
Teacher Epoch 19/20: 100% ██████████ 391/391 [00:52<00:00, 7.51batch/s, loss=1.4353]
Epoch 19: Val Acc 50.65%
Teacher Epoch 20/20: 100% ██████████ 391/391 [00:51<00:00, 7.62batch/s, loss=1.3627]
Epoch 20: Val Acc 51.07%

```

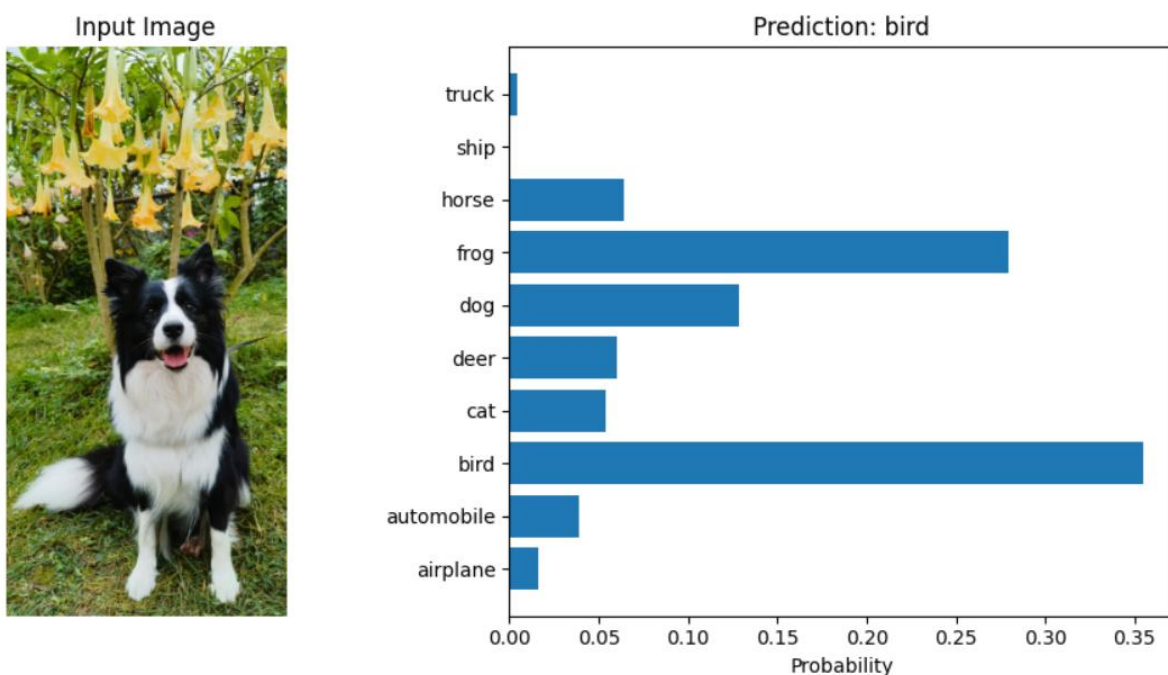
以下是单独训练的学生模型的单图预测结果，可以看到各个维度的特征概率分布非常不均匀，并且最终的预测标签也是错误的。

```

img_path = "data\dog.jpg" # 图片相对路径
prediction = predict_single_image(baseline_student, img_path)
print(f"最终预测结果: {prediction}")

```

[135] ✓ 0.2s



（四）对比评估一方案二

1. 整体对比

编写评估模型代码：


```

def generate_report(model, loader, classes):
    model.eval()
    all_labels = []
    all_preds = []

    with torch.no_grad():
        for images, labels in loader:
            outputs = model(images)
            _, preds = torch.max(outputs, 1)
            all_labels.extend(labels.numpy())
            all_preds.extend(preds.numpy())

    # 分类报告
    print("\nClassification Report:")
    print(classification_report(all_labels, all_preds, target_names=classes))

    # 混淆矩阵
    cm = confusion_matrix(all_labels, all_preds)
    plt.figure(figsize=(12, 10))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=classes, yticklabels=classes)
    plt.title("Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("True")
    #plt.savefig("confusion_matrix.png", dpi=150)
    plt.close()

```

✓ 0.0s

可以看出教师模型达到收敛需要的训练轮数最少，但训练时长最长，这是由模型复杂性决定的。此外，尽管知识蒸馏的学生模型训练时长较长，但准确率性能比单独训练的学生模型要更好一些。这里没有高太多可能是因为学生模型结构简单导致准确率受限。

	训练轮数	训练时长	准确率
教师模型	10 次	92min	84%
学生模型（知识蒸馏）	20 次	52min	52%
学生模型（单独训练）	20 次	16min	51%

以下分别是教师模型、知识蒸馏的学生模型、基线学生模型的 `classification_report`，可以看到在不同分类维度上，知识蒸馏的学生模型的准确率是比较接近教师模型的：教师模型准确率高的分类维度，学生模型的准确率也比较高。

=== Teacher Performance ===

Classification Report:

	precision	recall	f1-score	support
airplane	0.81	0.87	0.84	1000
automobile	0.88	0.95	0.92	1000
bird	0.73	0.86	0.79	1000
cat	0.68	0.74	0.71	1000
deer	0.82	0.85	0.83	1000
dog	0.90	0.64	0.75	1000
frog	0.85	0.90	0.87	1000
horse	0.88	0.86	0.87	1000
ship	0.93	0.90	0.91	1000
truck	0.95	0.80	0.87	1000
accuracy			0.84	10000
macro avg	0.84	0.84	0.84	10000
weighted avg	0.84	0.84	0.84	10000

=== Distilled Student Performance ===

Classification Report:

	precision	recall	f1-score	support
airplane	0.53	0.51	0.52	1000
automobile	0.60	0.69	0.64	1000
bird	0.40	0.47	0.43	1000
cat	0.37	0.24	0.29	1000
deer	0.48	0.47	0.47	1000
dog	0.46	0.48	0.47	1000
frog	0.56	0.59	0.58	1000
horse	0.55	0.60	0.57	1000
ship	0.59	0.64	0.61	1000
truck	0.63	0.53	0.57	1000
accuracy			0.52	10000
macro avg	0.52	0.52	0.52	10000
weighted avg	0.52	0.52	0.52	10000

=== Baseline Student Performance ===

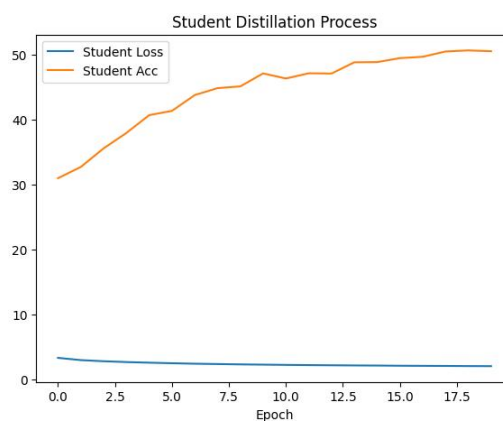
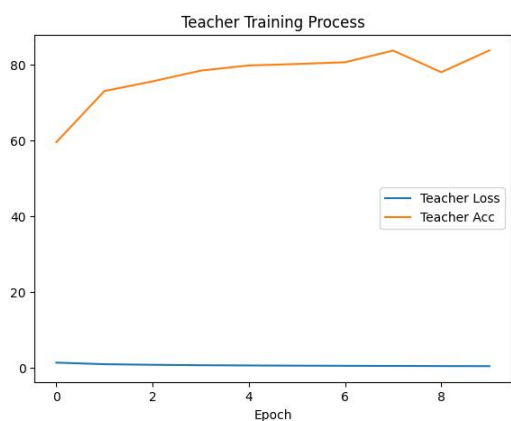
Classification Report:

	precision	recall	f1-score	support
airplane	0.54	0.53	0.54	1000
automobile	0.67	0.50	0.57	1000
bird	0.40	0.36	0.38	1000
cat	0.32	0.45	0.37	1000
deer	0.47	0.40	0.43	1000
dog	0.51	0.37	0.43	1000
frog	0.54	0.71	0.61	1000
horse	0.61	0.52	0.56	1000
ship	0.57	0.70	0.63	1000
truck	0.57	0.57	0.57	1000
accuracy			0.51	10000
macro avg	0.52	0.51	0.51	10000
weighted avg	0.52	0.51	0.51	10000

2. 教师模型与知识蒸馏学生模型

以下是对模型训练过程中的每一个 epoch 的损失值和准确率的可视化,由于学生模型比较简单,所以在实验中我增加了学生模型的训练轮数到 20 次。

由于电脑计算能力有限,所以使用的学生模型比较简单,模型性能有限,准确率和教师模型相差较大。



（五）超参数对模型效果的影响总结

$T > 1$: 增加温度, 使得 softmax 分布变得 更平滑, 所有类别的概率差距减小。 $T < 1$: 降低温度, 使得 softmax 分布 更尖锐, 接近 one-hot 分布。当 T 增大时, 模型输出会更加丰富, 学生模型能够学习到教师模型的隐含知识。

α 接近 1 时, 更依赖教师模型的软标签; α 接近 0 时, 更依赖原始真实标签。

在最初实验时, 我将这两个关键的超参数设置为: Temperature=5, $\alpha=0.8$ 。发现训练的学生模型准确率仅不到 40%, 如图:

```
> ~
# 实验2: 知识蒸馏训练学生
print("\n=== Distilling Student ===")
teacher.load_state_dict(torch.load("models/best_teacher.pth"))
student = Student()
student_hist = distill(student, teacher, train_loader, test_loader)

[90] ✓ 42m 27.7s
```

```
...
=== Distilling Student ===
Distill Epoch 1/20: 100%|██████████| 391/391 [01:50<00:00, 3.55batch/s, loss=7.1835, acc=21.14%]
Epoch 1: Val Acc 23.33%
Distill Epoch 2/20: 100%|██████████| 391/391 [01:41<00:00, 3.85batch/s, loss=5.7867, acc=26.65%]
Epoch 2: Val Acc 27.32%
Distill Epoch 3/20: 100%|██████████| 391/391 [01:39<00:00, 3.94batch/s, loss=4.6949, acc=29.13%]
Epoch 3: Val Acc 29.38%
Distill Epoch 4/20: 100%|██████████| 391/391 [01:48<00:00, 3.61batch/s, loss=4.6479, acc=30.62%]
Epoch 4: Val Acc 30.90%
Distill Epoch 5/20: 100%|██████████| 391/391 [01:39<00:00, 3.92batch/s, loss=4.4218, acc=32.13%]
Epoch 5: Val Acc 32.68%
Distill Epoch 6/20: 100%|██████████| 391/391 [01:38<00:00, 3.96batch/s, loss=4.1214, acc=33.14%]
Epoch 6: Val Acc 33.31%
Distill Epoch 7/20: 100%|██████████| 391/391 [01:39<00:00, 3.95batch/s, loss=4.3386, acc=33.89%]
Epoch 7: Val Acc 33.95%
Distill Epoch 8/20: 100%|██████████| 391/391 [01:39<00:00, 3.91batch/s, loss=4.6542, acc=34.76%]
Epoch 8: Val Acc 34.78%
Distill Epoch 9/20: 100%|██████████| 391/391 [01:39<00:00, 3.95batch/s, loss=4.0835, acc=35.41%]
Epoch 9: Val Acc 36.42%
Distill Epoch 10/20: 100%|██████████| 391/391 [01:39<00:00, 3.94batch/s, loss=3.4031, acc=36.32%]
Epoch 10: Val Acc 36.15%
Distill Epoch 11/20: 100%|██████████| 391/391 [01:39<00:00, 3.93batch/s, loss=3.9545, acc=36.59%]
Epoch 11: Val Acc 37.54%
Distill Epoch 12/20: 100%|██████████| 391/391 [01:42<00:00, 3.81batch/s, loss=4.1807, acc=37.32%]
Epoch 12: Val Acc 37.36%
Distill Epoch 13/20: 100%|██████████| 391/391 [01:40<00:00, 3.90batch/s, loss=3.8758, acc=37.67%]
```

随后为了提升模型性能, 因为目前模型存在欠拟合问题, 所以需要降低 T 值, 使输出更加固定, 减少多样化带来的概率分配不准确问题。因此, 我尝试降低了 T 和 α 分别为 (3 和 0.5), 使模型输出分布概率更尖锐, 更依据于原始分部标签, 性能有了显著提高。

（六）错误分类案例输出

首先编写错误分类函数, 调整模型为评估模式, 收集分类错误的样本:

```
def analyze_misclassified(model, test_loader, class_names, num_samples=9):
    model.eval()
    misclassified = []

    with torch.no_grad():
        for images, labels in test_loader:
            outputs = model(images)
            _, preds = torch.max(outputs, 1)

            # 收集错误样本
            mask = (preds != labels)
            misclassified.extend([
                (images[i], labels[i], preds[i])
                for i in range(len(labels)) if mask[i]
            ])

            if len(misclassified) >= num_samples:
                break
```

将错误分类的图片进行可视化展示时，进行反归一化处理：

```
# 可视化展示
plt.figure(figsize=(12, 12))
for idx in range(min(num_samples, len(misclassified))):
    img, true_label, pred_label = misclassified[idx]

    # 反归一化处理
    img = img.numpy().transpose((1, 2, 0))
    mean = np.array([0.4914, 0.4822, 0.4465])
    std = np.array([0.2023, 0.1994, 0.2010])
    img = std * img + mean
    img = np.clip(img, 0, 1)

    plt.subplot(3, 3, idx+1)
    plt.imshow(img)
    plt.title(f"True: {class_names[true_label]}\nPred: {class_names[pred_label]}")
    plt.axis('off')

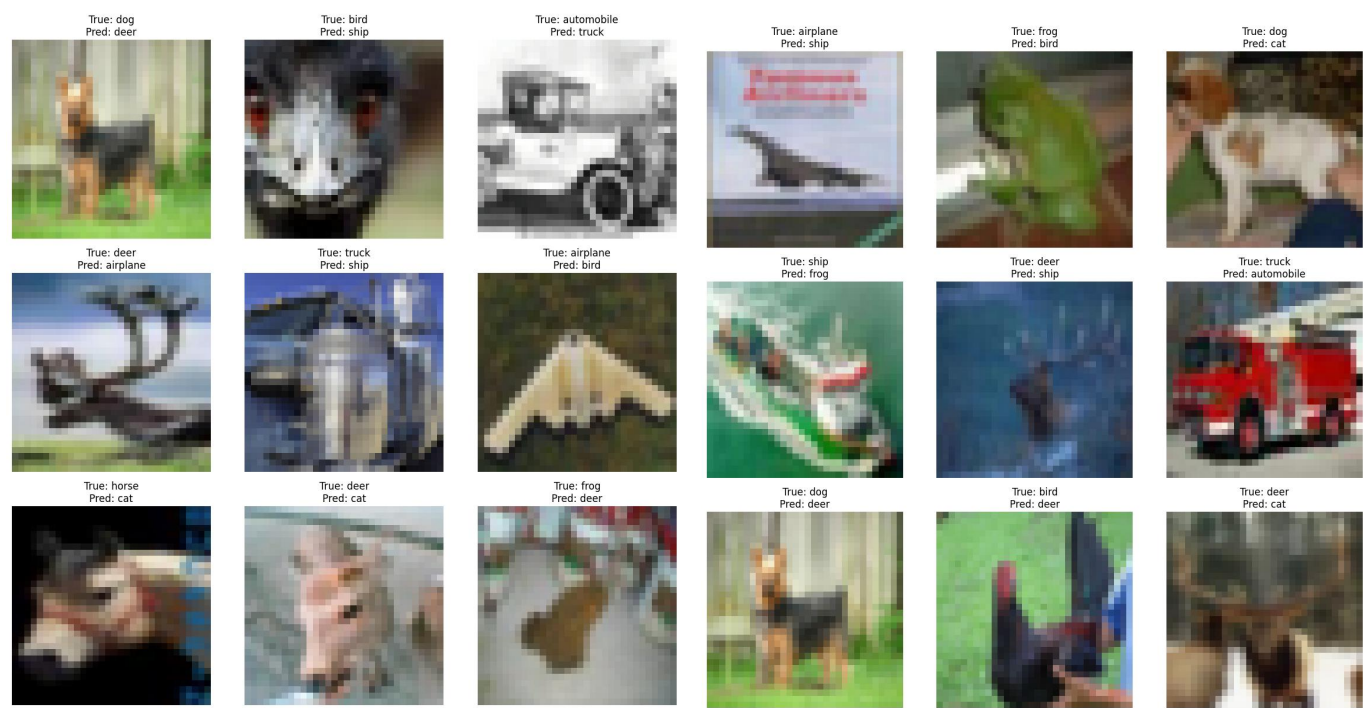
plt.tight_layout()
plt.show()
```

调用函数：

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']
analyze_misclassified(teacher, test_loader, class_names, num_samples=9)
```

8] ✓ 11.8s

只输出 9 个错误分类案例，教师模型和知识蒸馏训练学生模型的分错误实例如下：



单独训练的学生模型的错误分类实例如下：

